MESTRADO EM ENGENHARIA INFORMÁTICA

Tecnologias de Redes de Comunicações Móveis

TRABALHO LABORATORIAL 6

Simulação de Monte Carlo da Probabilidade de Colisão de IV no WEP

Paradoxo do aniversário, validação numérica e comparação com o TKIP



Autor

Claudio Baptista

Data

Outubro de 2026

Implementação

Python 3 · NumPy · Matplotlib

Resumo

Este trabalho estima, por simulação de Monte Carlo, a probabilidade de dois pacotes cifrados com WEP reutilizarem o mesmo vetor de inicialização (IV) de 24 bits, em função do número de pacotes transmitidos, e compara-a com a fórmula exata do problema do aniversário generalizado. Foram implementados em Python dois estimadores independentes — um *direto*, com 5 000 ensaios por ponto, e um baseado no instante da *primeira colisão*, com 200 000 ensaios — que concordam entre si e com a teoria dentro da incerteza estatística esperada. A validação inclui ainda a comparação de três geradores pseudoaleatórios (Philox, MT19937 e PCG64).

Para n = 4 823 pacotes a probabilidade teórica de colisão é 50,001%, e a simulação obteve 50,01% (IC 95%: 49,79% a 50,23%), confirmando o resultado do Exercício 1. Repetindo a análise para o IV de 48 bits do TKIP, são necessários 19 753 663 pacotes para atingir os mesmos 50%: um fator de 4 096 ($= 2^{12} = \sqrt{(2^{48}/2^{24})}$). Com os mesmos 4 823 pacotes, a probabilidade de colisão no TKIP desce para $4{,}13 \times 10^{-8}$, cerca de $1{,}2 \times 10^{7}$ vezes menor. Um estudo de escala entre 16 e 48 bits confirma empiricamente a lei $n_{50} \propto \sqrt{N}$: duplicar o número de bits do IV apenas eleva ao quadrado — e não à potência do tamanho do espaço — a resistência a colisões.

O relatório termina com uma discussão sobre a razão pela qual o método de Monte Carlo, criado em Los Alamos na década de 1940, continua a ser uma ferramenta central em engenharia de telecomunicações, segurança, finanças, física, computação gráfica e inteligência artificial.

Palavras-chave: WEP, TKIP, vetor de inicialização, paradoxo do aniversário, método de Monte Carlo, segurança em redes sem fios.

Índice

Resumo . . . 2

1. Introdução . . . 4

2. Enquadramento: o IV no WEP e no TKIP . . . 4

   2.1 Construção da chave por pacote no WEP . . . 4

   2.2 Porque é que uma colisão de IV é grave . . . 4

   2.3 A resposta do WPA: o TKIP . . . 5

3. Modelo teórico: o problema do aniversário generalizado . . . 5

   3.1 Probabilidade exata de colisão . . . 5

   3.2 Aproximação exponencial e o valor n50 . . . 5

   3.3 Instante da primeira colisão . . . 6

4. O método de Monte Carlo . . . 6

   4.1 Princípio . . . 6

   4.2 Quantificação da incerteza . . . 7

   4.3 Origem histórica . . . 7

5. Implementação . . . 7

   5.1 Arquitetura . . . 7

   5.2 Estimador 1 — método direto . . . 8

   5.3 Estimador 2 — instante da primeira colisão . . . 8

   5.4 Plano experimental . . . 8

6. Resultados e discussão . . . 9

   6.1 WEP: teoria vs. simulação . . . 9

   6.2 Questão 1 — N ≈ 4 823 pacotes conduz a 50% de colisão? . . . 10

   6.3 Questão 2 — rapidez de crescimento: WEP (24 bits) vs. TKIP (48 bits) . . . 11

   6.4 Questão 3 — quantos pacotes para 50% com um IV de 48 bits? . . . 13

   6.5 Convergência e precisão do estimador . . . 14

   6.6 Validação do gerador pseudoaleatório . . . 15

   6.7 Síntese comparativa . . . 16

7. Porque é que o método de Monte Carlo continua tão importante . . . 16

   7.1 Razões estruturais . . . 16

   7.2 Aplicações atuais . . . 17

   7.3 Limitações e como são ultrapassadas . . . 17

8. Conclusões . . . 18

Referências . . . 18

Anexo A — Núcleo do estimador por primeira colisão . . . 20

Anexo B — Reprodução dos resultados . . . 20

1. Introdução

O *Wired Equivalent Privacy* (WEP), definido na norma IEEE 802.11 de 1997, foi o primeiro mecanismo de confidencialidade das redes Wi-Fi. A sua fragilidade mais conhecida não está no comprimento da chave, mas na forma como cada pacote obtém a sua própria chave de cifra: concatenando uma chave secreta fixa com um **vetor de inicialização (IV) de apenas 24 bits**, transmitido em claro. Sempre que dois pacotes usam o mesmo IV, usam também a mesma sequência de cifra RC4 — e isso abre a porta a ataques.

O objetivo deste trabalho, conforme o enunciado do Trabalho 6, é **simular por Monte Carlo a probabilidade de colisão de IV no WEP em função do número de pacotes transmitidos** e compará-la com a aproximação teórica do paradoxo do aniversário. Concretamente:

- simular a geração aleatória de IVs num espaço de 2^{24} valores e estimar empiricamente a fração de simulações com pelo menos uma colisão;
- comparar a curva simulada com a fórmula teórica exata do problema do aniversário generalizado;
- repetir a análise para um IV de 48 bits, como no TKIP do WPA, e comparar a robustez face a colisões.

Para além das três questões de análise do enunciado, o trabalho inclui um estudo da convergência do estimador (erro $\propto 1/\sqrt{T}$), a distribuição completa do instante da primeira colisão e uma verificação empírica da lei de escala $n_{50} \propto \sqrt{N}$ para IVs entre 16 e 48 bits. O Capítulo 2 enquadra o problema de segurança; o Capítulo 3 desenvolve o modelo teórico; o Capítulo 4 apresenta o método de Monte Carlo; o Capítulo 5 descreve a implementação; o Capítulo 6 apresenta e compara os resultados; o Capítulo 7 discute a relevância atual do método; e o Capítulo 8 conclui.

2. Enquadramento: o IV no WEP e no TKIP

2.1 Construção da chave por pacote no WEP

O WEP usa a cifra de fluxo RC4. Para cada pacote, o emissor escolhe um IV de 24 bits e forma a semente RC4 concatenando-o com a chave partilhada (40 ou 104 bits, dando as variantes comerciais «64» e «128 bits»). O RC4 gera uma sequência pseudoaleatória (*keystream*) que é combinada por XOR com os dados e com o CRC-32 (ICV). O IV segue em claro no cabeçalho, para que o recetor possa reconstruir a mesma semente.

$$C = (P \| \mathrm{ICV}(P)) \oplus \mathrm{RC4}(\mathrm{IV} \| K)$$

2.2 Porque é que uma colisão de IV é grave

Como a chave K é fixa durante longos períodos, a *keystream* depende apenas do IV. Se dois pacotes P_1 e P_2 forem cifrados com o mesmo IV, então $C_1 \oplus C_2 = P_1 \oplus P_2$: a cifra anula-se e o atacante obtém o XOR dos dois textos em claro. Com um texto conhecido (por exemplo, um cabeçalho ARP ou IP previsível), recupera o outro, e pode construir progressivamente uma tabela de *keystreams* indexada por IV. Este problema foi descrito por Borisov, Goldberg e Wagner em 2001 e, no mesmo ano, Fluhrer, Mantin e Shamir mostraram que certos IVs revelam também bits da própria chave RC4 (ataque FMS), culminando em ataques práticos que recuperam uma chave WEP de 104 bits com

algumas dezenas de milhar de pacotes (Tews, Weinmann e Pyshkin, 2007).

A pergunta central deste trabalho é, portanto: **quantos pacotes é preciso observar até que a reutilização de um IV se torne provável?** A intuição de que $2^{24} \approx 16{,}8$ milhões de IVs «chegam para muito tempo» é enganadora — é precisamente o paradoxo do aniversário.

2.3 A resposta do WPA: o TKIP

O TKIP (*Temporal Key Integrity Protocol*), introduzido com o WPA em 2003 como solução de transição que pudesse correr no hardware WEP existente, alarga o IV para um contador de sequência de **48 bits** (TSC), aplica uma função de mistura que gera uma chave RC4 diferente para cada pacote, acrescenta o código de integridade Michael e renova periodicamente as chaves temporais. Neste trabalho isolamos apenas o efeito do aumento do espaço de IV de 2^{24} para 2^{48}, assumindo — tal como para o WEP — IVs escolhidos aleatoriamente. É um cenário conservador: no TKIP o TSC é sequencial e nunca se repete dentro da mesma chave temporal, pelo que o modelo aleatório representa o pior caso. O TKIP é hoje considerado obsoleto (foi desaconselhado na revisão IEEE 802.11-2012), tendo sido substituído pelo CCMP/AES do WPA2 e pelo GCMP do WPA3.

3. Modelo teórico: o problema do aniversário generalizado

3.1 Probabilidade exata de colisão

Seja $N = 2^b$ o número de IVs possíveis ($b = 24$ no WEP, $b = 48$ no TKIP) e suponha-se que cada um de n pacotes recebe um IV uniforme e independente. O primeiro pacote nunca colide; o segundo é distinto do primeiro com probabilidade $(N-1)/N$; o k-ésimo é distinto de todos os anteriores com probabilidade $(N-k+1)/N$. Assim, a probabilidade de *não* haver nenhuma colisão é um produto, e a de haver pelo menos uma é o seu complemento:

$$P(n) \;=\; 1 - \prod_{k=0}^{n-1} \left(1 - \frac{k}{N}\right) \;=\; 1 - \frac{N!}{(N-n)!\; N^{\,n}} \tag{1}$$

A Equação (1) é exata e é a referência usada em todas as comparações. Para $N = 2^{48}$ o produto tem dezenas de milhões de fatores muito próximos de 1, pelo que foi avaliado em domínio logarítmico, como soma de $\log(1-k/N)$, de forma a evitar perda de precisão numérica.

3.2 Aproximação exponencial e o valor n_{50}

Usando $1-x \approx e^{-x}$ para x pequeno, a soma dos expoentes é $n(n-1)/2N$, o que dá a aproximação clássica e, invertendo-a, o número de pacotes para uma probabilidade-alvo p:

$$P(n) \;\approx\; 1 - e^{-n(n-1)/2N}, \qquad n_p \;\approx\; \sqrt{2N \ln \frac{1}{1-p}} \tag{2}$$

$$n_{50} \;\approx\; \sqrt{2\ln 2}\,\sqrt{N} \;\approx\; 1{,}1774\,\sqrt{N} \tag{3}$$

Para o WEP, $\sqrt{(2^{24})} = 4\,096$ e $n_{50} \approx 4\,822{,}7$, ou seja, **cerca de 4 823 pacotes** — o valor do Exercício 1. A procura exata (menor n com $P(n) \geq 0{,}5$ na Equação 1) dá também $n_{50} = 4\,823$. Para o TKIP obtém-se $n_{50} = 19\,753\,663$.

Uma consequência imediata, e talvez a mais importante do trabalho, é que n_{50} cresce com $\sqrt{N}$ e não com N: duplicar o número de bits do IV não duplica a segurança, eleva-a ao quadrado. Passar de 24 para 48 bits multiplica N por 2^{24}, mas multiplica n_{50} apenas por 2^{12} = 4 096.

A Tabela 1 compara a fórmula exata com a aproximação exponencial e com o majorante «quadrático» n(n−1)/2N (desigualdade de Boole), por vezes usado como aproximação grosseira. A aproximação exponencial é excelente em toda a gama; o majorante só é útil para probabilidades pequenas e chega a ultrapassar 1, o que mostra que não deve ser usado perto do ponto dos 50%.

***Tabela 1** — Fórmula exata vs. aproximações para o WEP (N = 2^{24}).*

n (pacotes)	Exata (Eq. 1)	Aprox. exponencial	Majorante n(n−1)/2N	Erro da aprox. exp.
1 000	2,933%	2,933%	3,0%	-0,0001 p.p.
2 000	11,233%	11,233%	11,9%	-0,0004 p.p.
4 823	50,001%	49,998%	69,3%	-0,0033 p.p.
10 000	94,923%	94,920%	2,98 (> 1!)	-0,0030 p.p.
20 000	99,999%	99,999%	11,92 (> 1!)	-0,0000 p.p.

3.3 Instante da primeira colisão

Em vez de fixar n, pode perguntar-se: *em que pacote K ocorre a primeira colisão?* K = k significa que os primeiros k−1 IVs são distintos e o k-ésimo repete um deles, donde:

$$\Pr[K=k] \;=\; (1-P(k-1))\tfrac{k-1}{N}, \qquad \Pr[K\le n] \;=\; P(n) \tag{4}$$

$$\mathbb{E}[K] \;\approx\; \sqrt{\tfrac{\pi N}{2}} + \tfrac{2}{3}, \qquad \sigma_K \;\approx\; \sqrt{(2-\tfrac{\pi}{2})N} \tag{5}$$

A segunda igualdade da Equação (4) é a base do segundo estimador de Monte Carlo usado neste trabalho. Para o WEP, E[K] ≈ 5 134 e σ_K ≈ 2 683 pacotes: a mediana (≈ 4 823) é inferior à média porque a distribuição tem cauda à direita.

4. O método de Monte Carlo

4.1 Princípio

O método de Monte Carlo estima uma grandeza determinística — uma probabilidade, um integral, um valor esperado — através da média de muitas experiências aleatórias independentes. Se cada ensaio i produz um indicador $X_i \in \{0,1\}$ que vale 1 quando ocorre o acontecimento de interesse (aqui, «houve pelo menos uma colisão de IV»), então a fração empírica (designada P_{sim} no resto do relatório) é um estimador não enviesado de P:

$$\hat{P}_T \;=\; \tfrac{1}{T}\sum_{i=1}^{T} X_i, \qquad \mathbb{E}[\hat{P}_T] = P, \qquad \sigma_{\hat{P}} \;=\; \sqrt{\tfrac{P(1-P)}{T}} \tag{6}$$

A lei dos grandes números garante que $P_{sim} \to P$ quando $T \to \infty$, e o teorema do limite central diz-nos *quão depressa*: o erro decresce como $1/\sqrt{T}$. Reduzir o erro 10 vezes exige 100 vezes mais

ensaios. Esta taxa é independente da dimensão do problema — característica que, como se verá no Capítulo 7, explica grande parte da importância atual do método.

4.2 Quantificação da incerteza

Cada estimativa é acompanhada de um intervalo de confiança a 95%. Usou-se o intervalo de Wilson (1927), preferível ao intervalo normal simples porque se mantém válido quando P_{sim} está próximo de 0 ou de 1 (situação frequente nas caudas das curvas de colisão e, sobretudo, no caso TKIP):

$$\mathrm{IC}_{95\%} = \frac{\hat{P} + z^2/2T \pm z\sqrt{\hat{P}(1-\hat{P})/T + z^2/4T^2}}{1 + z^2/T}, \qquad z = 1{,}96 \tag{7}$$

4.3 Origem histórica

O método foi formulado em 1946–1947 em Los Alamos por Stanislaw Ulam e John von Neumann, no contexto do estudo da difusão de neutrões, e batizado por Nicholas Metropolis em alusão ao casino do Mónaco. Ulam contou que a ideia lhe surgiu ao tentar calcular a probabilidade de ganhar um jogo de paciência: era mais fácil jogar muitas vezes e contar do que resolver a combinatória. As primeiras simulações correram no ENIAC em 1948, e o artigo fundador, «The Monte Carlo Method», de Metropolis e Ulam, foi publicado em 1949. Este trabalho segue exatamente essa filosofia: o problema tem solução analítica, mas a simulação permite validá-la de forma independente e explorar variantes sem nova derivação matemática.

5. Implementação

5.1 Arquitetura

A implementação, em Python 3 com NumPy e Matplotlib, está organizada em quatro módulos com responsabilidades separadas (Tabela 2). Todo o código está em `src/`, as figuras em `figuras/` e os resultados numéricos (JSON, CSV e as amostras brutas em NPY) em `resultados/`.

***Tabela 2** — Módulos da implementação.*

Módulo	Responsabilidade
`teoria.py`	Fórmula exata (domínio logarítmico), aproximações, inversão para n_p (pesquisa binária exata), distribuição, média e desvio-padrão de K.
`montecarlo.py`	Os dois estimadores de Monte Carlo, intervalo de Wilson, paralelização em vários processos com sementes independentes.
`experiencias.py`	Orquestra as experiências E1–E6, mede tempos, grava resultados e gera as figuras. Opções `--rapido` e `--so-figuras`.
`graficos.py`	Figuras com convenção visual única: WEP a azul, TKIP a laranja, teoria a traço contínuo escuro, simulação com marcadores e IC 95%.

5.2 Estimador 1 — método direto

É a tradução literal do enunciado. Para cada n, realizam-se T ensaios; em cada um geram-se n IVs uniformes em $[0, 2^b)$ e verifica-se se há repetidos. A verificação é feita por ordenação: depois de ordenar o vetor, existe colisão se e só se dois elementos consecutivos forem iguais — custo $O(n \log n)$ em vez dos $O(n^2)$ de comparar todos os pares. Os ensaios são vetorizados por lotes: uma matriz (lote × n) é ordenada linha a linha numa única chamada NumPy, evitando ciclos em Python.

5.3 Estimador 2 — instante da primeira colisão

Em cada ensaio geram-se IVs até à primeira repetição e regista-se o índice K. Como {colisão nos primeiros n pacotes} = {K ≤ n}, a função de distribuição empírica de K dá $P_{sim}(n)$ *para todos os n em simultâneo*, com uma só bateria de ensaios. É este estimador que torna viável o caso de 48 bits, em que cada ensaio envolve dezenas de milhões de IVs.

Três decisões de implementação tornaram-no eficiente:

- **Geração em bloco.** Em vez de inserir IVs um a um num conjunto (lento em Python), gera-se de uma vez um bloco de $L = 3\sqrt{N}$ IVs, que contém uma colisão com probabilidade $1-e^{-4,5} \approx 98,9\%$. Se não contiver, o bloco é prolongado; o resultado é sempre exato.
- **Localização sem *argsort*.** Ordenar índices de 50 milhões de elementos é cerca de 25 vezes mais lento do que ordenar os valores. Ordena-se apenas uma cópia dos valores, extraem-se os poucos valores repetidos (≈4,5 em média) e só então se procuram as suas posições no vetor original; a primeira posição cujo valor já apareceu é K.
- **Paralelização reprodutível.** Os ensaios são distribuídos por todos os núcleos com `ProcessPoolExecutor`; cada tarefa recebe uma semente filha de `numpy.random.SeedSequence`, o que garante fluxos aleatórios independentes e resultados idênticos em execuções repetidas. O gerador usado é o **Philox**, um gerador baseado em contador concebido para simulação paralela (a escolha é justificada na secção 6.6).

O núcleo do estimador encontra-se no Anexo A. Os dois estimadores são independentes — usam sementes e algoritmos diferentes —, pelo que a sua concordância constitui uma validação cruzada da implementação.

5.4 Plano experimental

Tabela 3 — Experiências realizadas (tempo total: 17,2 min num Ryzen 7 3700X, 8 núcleos).

Exp.	Descrição	Ensaios	Tempo
E1	WEP, método direto, 12 valores de n entre 500 e 15 000	5 000 por ponto	9 s
E2	WEP, primeira colisão (curva completa, distribuição de K)	200 000	9 s
E3	TKIP, primeira colisão	500	530 s
E4	TKIP com os mesmos n do E1, método direto	5 000 por ponto	14 s
E5	Convergência do estimador (reutiliza as amostras do E2)	—	< 1 s
E6	Lei de escala, IV de 16 a 48 bits (passo 4)	500 a 200 000	411 s
E7	Validação do gerador: WEP com Philox, MT19937 e PCG64; N = 2^{10} com Philox	300 000 por gerador	38 s

6. Resultados e discussão

6.1 WEP: teoria vs. simulação

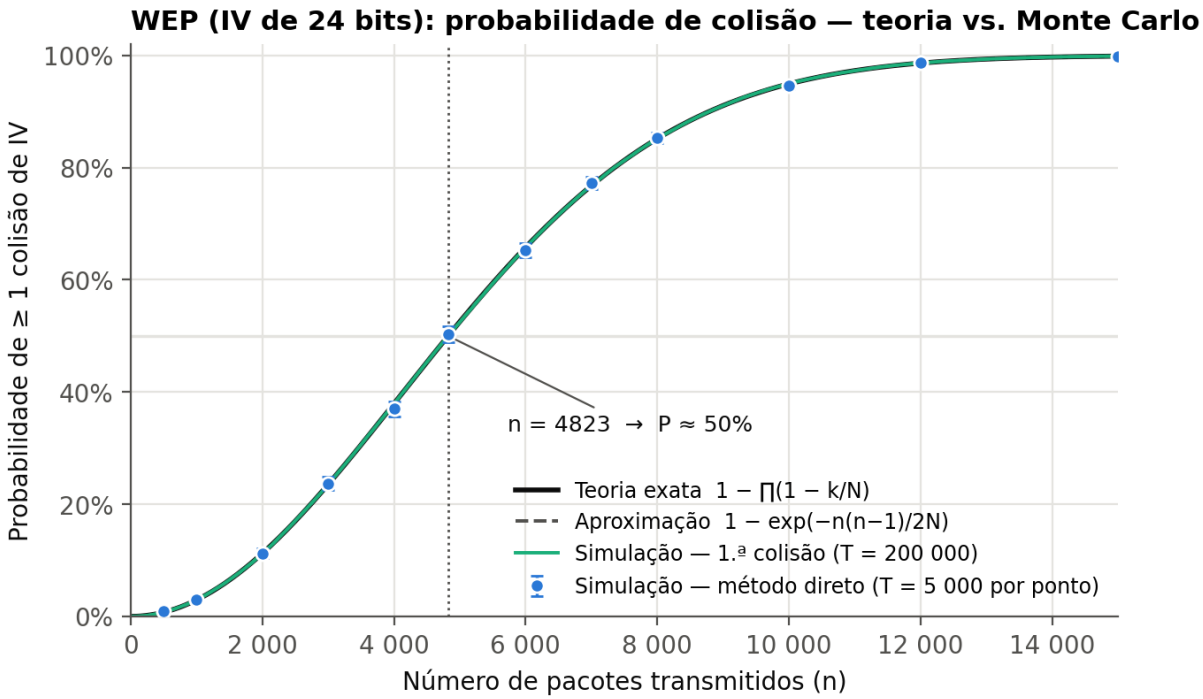


Figura 1 — Probabilidade de colisão de IV no WEP. Os pontos (método direto) e a curva verde (primeira colisão) são simulações; a curva preta é a Equação 1.

A Figura 1 sobrepõe os dois estimadores à fórmula exata. As três curvas são praticamente indistinguíveis em toda a gama, e a aproximação exponencial coincide com a exata à escala do gráfico. Dos 12 pontos do método direto, **12 têm o valor teórico dentro do respetivo intervalo de confiança a 95%**, o que é compatível com o nível de confiança nominal (esperar-se-iam, em média, 11,4). A Tabela 4 detalha os valores.

Tabela 4 — Resultados do método direto para o WEP (E1). «Desvio» = $(P_{sim} - P)/\sigma_{P_{sim}}$.

n	Colisões / ensaios	P_{sim} simulada	IC 95% (Wilson)	P exata	Desvio (σ)
500	44 / 5 000	0,88%	[0,66; 1,18]%	0,74%	1,15
1 000	149 / 5 000	2,98%	[2,54; 3,49]%	2,93%	0,20
2 000	559 / 5 000	11,18%	[10,34; 12,08]%	11,23%	-0,12
3 000	1 184 / 5 000	23,68%	[22,52; 24,88]%	23,52%	0,27
4 000	1 851 / 5 000	37,02%	[35,69; 38,37]%	37,92%	-1,31
4 823	2 514 / 5 000	50,28%	[48,89; 51,67]%	50,00%	0,39
6 000	3 264 / 5 000	65,28%	[63,95; 66,59]%	65,80%	-0,77
7 000	3 864 / 5 000	77,28%	[76,10; 78,42]%	76,78%	0,83
8 000	4 266 / 5 000	85,32%	[84,31; 86,27]%	85,15%	0,33
10 000	4 730 / 5 000	94,60%	[93,94; 95,19]%	94,92%	-1,04
12 000	4 936 / 5 000	98,72%	[98,37; 99,00]%	98,63%	0,53
15 000	4 994 / 5 000	99,88%	[99,74; 99,94]%	99,88%	0,04

6.2 Questão 1 — N ≈ 4 823 pacotes conduz a 50% de colisão?

Sim. Para n = 4 823: P exata = **50,0009%**; aproximação exponencial = 49,9975%; método direto = **50,28%** (IC 95% [48,89%; 51,67%], 5 000 ensaios); primeira colisão = **50,01%** (IC 95% [49,79%; 50,23%], 200 000 ensaios). A mediana empírica de K é 4 823 pacotes, contra 4 823 teóricos.

Ambos os intervalos de confiança contêm o valor exato, e o estimador com mais ensaios fica a 0,01 pontos percentuais do valor exato. Em termos práticos: **num ponto de acesso WEP, bastam cerca de 4 800 pacotes — uma fração ínfima dos 16,8 milhões de IVs disponíveis (0,03%) — para que seja mais provável do que não ter já ocorrido uma reutilização de IV.** E com 3 × esse valor (≈15 000 pacotes), a colisão é praticamente certa (≈99,9%).

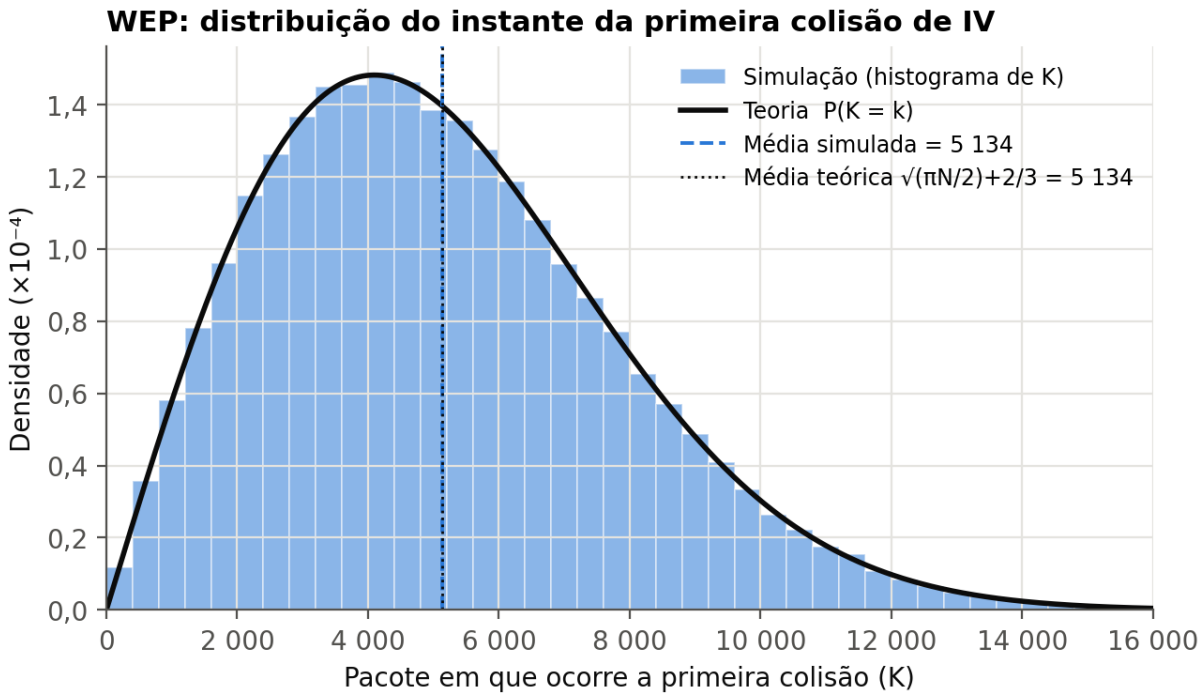


Figura 2 *— Histograma do pacote em que ocorre a primeira colisão (E2) e distribuição teórica Pr[K = k] (Equação 4).*

A Figura 2 mostra a distribuição do instante da primeira colisão. O histograma acompanha a densidade teórica, com média simulada 5 134 (teórica 5 134) e desvio-padrão 2 682 (teórico 2 683). Nos 200 000 ensaios, a colisão mais precoce ocorreu logo no pacote 9 e a mais tardia no pacote 20 022: mesmo no caso mais favorável observado, o WEP repetiu um IV antes de usar 0,12% do seu espaço de IVs.

6.3 Questão 2 — rapidez de crescimento: WEP (24 bits) vs. TKIP (48 bits)

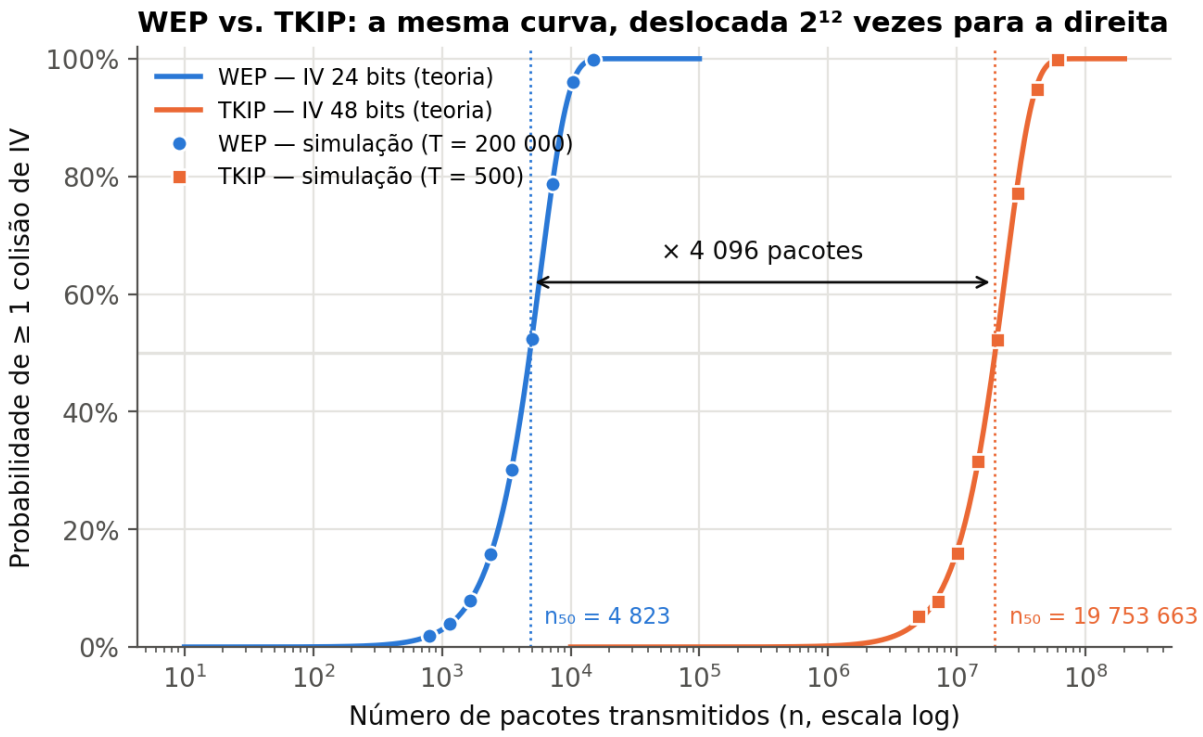


Figura 3 *— Probabilidade de colisão em função de n (escala logarítmica) para os dois tamanhos de IV, com pontos simulados (E2 e E3).*

Em escala logarítmica (Figura 3), as duas curvas têm exatamente a mesma forma: a do TKIP é a do WEP deslocada por um fator constante de 2^{12}. Os pontos simulados do TKIP (500 ensaios de primeira colisão, cada um com dezenas de milhões de IVs de 48 bits) seguem a curva teórica, com a dispersão esperada para essa dimensão de amostra.

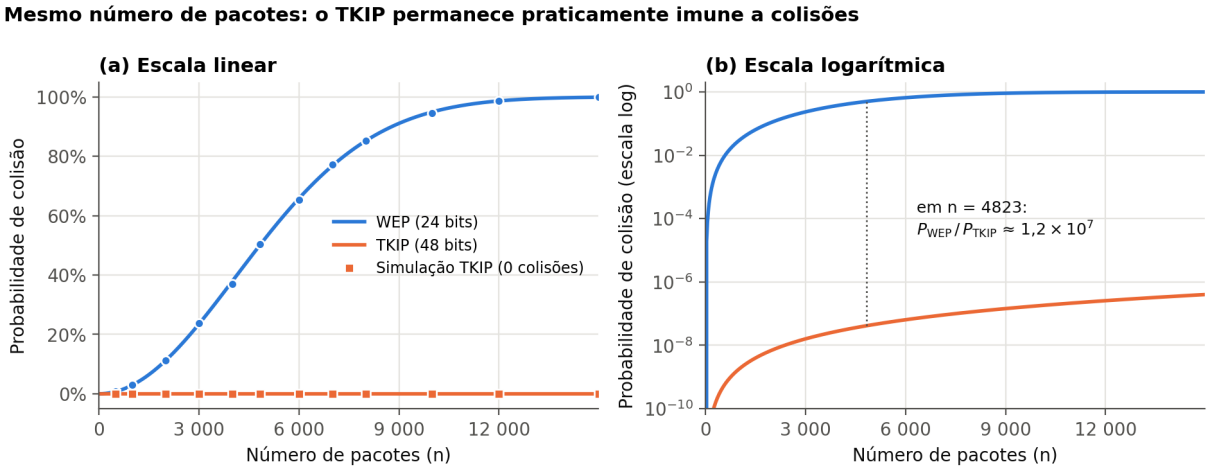


Figura 4 *— Mesmo número de pacotes para os dois protocolos: (a) escala linear, com pontos simulados (E1 e E4); (b) escala logarítmica da probabilidade.*

Para o mesmo número de pacotes (Figura 4), a diferença é de muitas ordens de grandeza. Enquanto a curva do WEP sobe de quase 0 a quase 100% em 15 000 pacotes, a do TKIP é indistinguível de zero em escala linear. Na simulação E4, em **60 000 ensaios TKIP com n até 15 000 não ocorreu nenhuma colisão** (0 observadas), o que está de acordo com a teoria: o número esperado de ensaios com colisão era apenas $5{,}96 \times 10^{-3}$. A Tabela 5 resume a comparação.

***Tabela 5** — Probabilidade de colisão para o mesmo n (simulações: 200 000 ensaios WEP, 500 TKIP).*

n (pacotes)	WEP exata	WEP simulada	TKIP exata	TKIP simulada
1 000	2,93%	2,95%	$1{,}77 \times 10^{-9}$	0
2 000	11,23%	11,20%	$7{,}10 \times 10^{-9}$	0
4 823	50,00%	50,01%	$4{,}13 \times 10^{-8}$	0
10 000	94,92%	94,97%	$1{,}78 \times 10^{-7}$	0
20 000	≈ 100%	≈ 100%	$7{,}11 \times 10^{-7}$	0
100 000	≈ 100%	≈ 100%	$1{,}78 \times 10^{-5}$	0
1 000 000	≈ 100%	≈ 100%	0,18%	0,60%
19 753 663	≈ 100%	≈ 100%	50,00%	48,80%
50 000 000	≈ 100%	≈ 100%	98,82%	99,20%

Note-se que, para o TKIP com n pequeno, a simulação devolve 0 — não porque a probabilidade seja nula, mas porque é demasiado pequena para ser observada com qualquer número razoável de ensaios. Para estimar $P \approx 4{\times}10^{-8}$ com 10% de erro relativo seriam precisos cerca de $2{,}4{\times}10^{9}$ ensaios. É uma limitação conhecida do Monte Carlo simples — os *acontecimentos raros* — e é aqui que a fórmula analítica (ou técnicas como a amostragem por importância) se torna indispensável.

6.4 Questão 3 — quantos pacotes para 50% com um IV de 48 bits?

> Com um IV de 48 bits são necessários **19 753 663 pacotes** (fórmula exata) para uma probabilidade de colisão de 50%, contra 4 823 no WEP — um fator de **$4\,095{,}7 \approx 2^{12}$**. Por simulação (E3), a mediana de K foi 20 186 196 e P_{sim}(colisão em 19 753 663 pacotes) = 48,8% (IC 95% [44,4%; 53,2%]).

A melhoria é real, mas é importante interpretá-la corretamente. O espaço de IV cresceu $2^{24} \approx 16{,}8$ milhões de vezes, e no entanto a resistência a colisões aleatórias cresceu «apenas» 4 096 vezes — a raiz quadrada. A Tabela 6 traduz estes números em tempo, para diferentes cargas de tráfego.

***Tabela 6** — Tempo até 50% de probabilidade de colisão (IVs aleatórios) e até esgotar o espaço de IV (IVs sequenciais).*

Tráfego (pacotes/s)	WEP: 50% colisão	TKIP: 50% colisão	WEP: esgotar IVs	TKIP: esgotar IVs
100	48,2 s	54,9 h	46,6 h	89 194 anos
500	9,6 s	11,0 h	9,3 h	17 839 anos
1 000	4,8 s	5,5 h	4,7 h	8 919 anos
5 000	1,0 s	1,1 h	0,9 h	1 784 anos

Com 1 000 pacotes por segundo — valor comum numa rede moderadamente carregada, ou facilmente induzido por um atacante que reinjete pedidos ARP —, o WEP atinge 50% de probabilidade de colisão em **menos de 5 segundos**, e mesmo com IVs sequenciais esgota o espaço em menos de 5 horas. O TKIP, no pior caso aleatório, leva cerca de 5,5 horas a chegar aos 50%; e como na realidade usa um contador sequencial que só se repetiria ao fim de 2^{48} pacotes (milhares

de anos a este ritmo) e renova as chaves temporais muito antes disso, a reutilização de IV deixa de ser um vetor de ataque prático. As vulnerabilidades posteriormente descobertas no TKIP (ex.: Beck–Tews, 2008) exploram outros aspetos do protocolo, não colisões de IV.

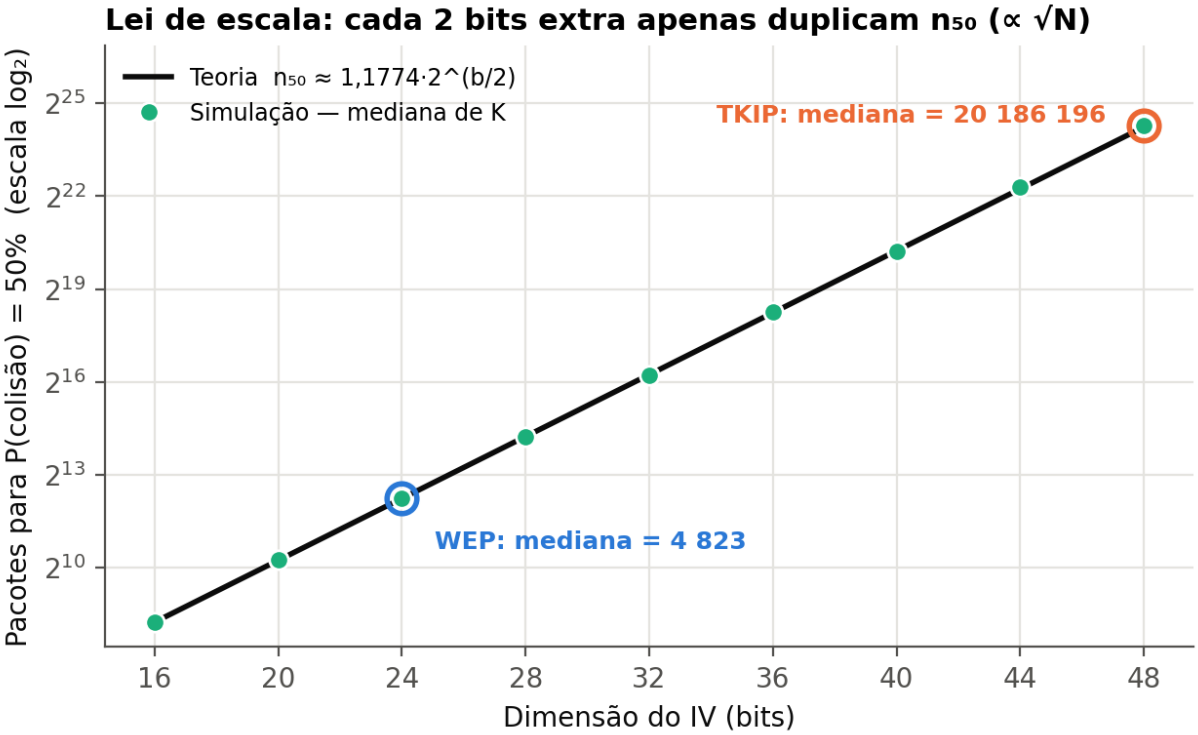


Figura 5 *— Lei de escala: mediana empírica de K (pontos) e n_{50} exato (linha) para IVs de 16 a 48 bits (E6). Eixo vertical em escala $\log_2$.*

A Figura 5 generaliza o resultado: para todos os tamanhos de IV simulados, a mediana empírica de K coincide com o n_{50} teórico (desvio relativo máximo de 3,4%, dominado pelos casos com menos ensaios). Em escala $\log_2$ obtém-se uma reta de declive ½: **cada bit adicional no IV multiplica n_{50} apenas por $\sqrt{2}$**. Esta é a razão pela qual, em criptografia, se exigem identificadores, *nonces* e resumos com o dobro dos bits da segurança pretendida (por exemplo, 256 bits de *hash* para 128 bits de resistência a colisões).

6.5 Convergência e precisão do estimador

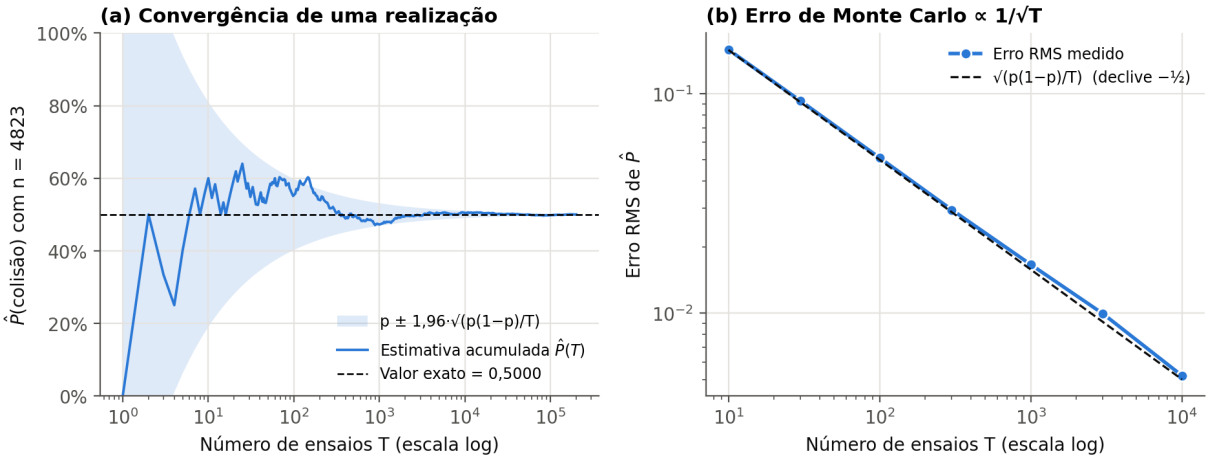


***Figura 6** — (a) Evolução da estimativa P_{sim} com o número de ensaios, dentro da banda de 95% prevista; (b) erro RMS medido vs. teórico $\sqrt{(P(1-P)/T)}$.*

A Figura 6 ilustra o comportamento estatístico do método. Em (a), com poucos ensaios a estimativa oscila fortemente, mas permanece dentro do funil de ±1,96σ e converge para o valor exato. Em (b), o erro RMS passa de 0,159 com T = 10 para 0,0052 com T = 10 000 — uma redução de cerca de 31 vezes para 1 000 vezes mais ensaios, tal como prevê a lei $1/\sqrt{T}$ (declive −½ em escala log-log; o ponto de maior T usa menos grupos independentes e tem, por isso, maior variabilidade). Este resultado justifica o dimensionamento das experiências: com 5 000 ensaios por ponto obtém-se um erro-padrão de ≈0,7 p.p. junto aos 50%, e com 200 000 ensaios de ≈0,1 p.p.

6.6 Validação do gerador pseudoaleatório

Um estudo de Monte Carlo é tão bom quanto o seu gerador de números aleatórios. Numa primeira execução completa, feita com o PCG64 (o gerador por omissão do NumPy), a média simulada de K para o WEP ficou 2,6 desvios-padrão acima do valor exato e $P_{sim}(4\,823)$ 2,3 desvios-padrão abaixo. Para despistar um erro de implementação repetiu-se o estimador em seis baterias independentes com PCG64 (1,2 milhões de ensaios, incluindo uma implementação ingénua IV a IV com um conjunto Python) e em 800 000 ensaios com outros dois geradores. O desvio manteve o mesmo sinal em todas as baterias PCG64 (significância conjunta ≈ 3σ para a média de K, correspondente a um desvio de ≈0,15% em P), mas não apareceu com Philox nem com MT19937 ($|z| < 0,5$), e o estimador reproduziu exatamente a média teórica para $N = 2^{10}$ e 2^{16}. A implementação ficou assim validada; quanto ao PCG64, a evidência não chega para afirmar que tem um defeito — pode tratar-se de uma flutuação —, mas, por precaução, todos os resultados finais deste relatório foram produzidos com o Philox. A experiência E7 deixa este teste registado e reprodutível (Tabela 7).

***Tabela 7** — Validação do gerador (E7): z = (simulado − exato)/erro-padrão; $|z| < 2$ é compatível com flutuação estatística a 95%.*

Gerador	N	Ensaios	E[K] simulado	E[K] exato	z (média)	z ($P(n_{50})$)
Philox	2^{10}	600 000	40,8	40,8	0,89	-0,04
Philox	2^{24}	300 000	5 127,0	5 134,2	-1,48	1,22
MT19937	2^{24}	300 000	5 132,6	5 134,2	-0,34	-0,29
PCG64	2^{24}	300 000	5 135,9	5 134,2	0,34	-0,90

Na execução final, os casos com Philox apresentam $|z| \leq 1{,}48$ — dentro do esperado para flutuações estatísticas. Nesta bateria, o PCG64 registou $z = 0{,}34$ para a média de K, um valor sem significado estatístico, o que reforça a hipótese de que o desvio inicial foi uma flutuação. A lição metodológica é geral: validar o gerador — comparando geradores independentes com um resultado exato conhecido — faz parte de qualquer estudo de Monte Carlo rigoroso.

6.7 Síntese comparativa

Tabela 8 *— Síntese: valores teóricos vs. simulados.*

Grandeza	Teoria	Simulação	Diferença
$P_{WEP}(4\,823)$ — método direto	50,00%	50,28%	0,28 p.p.
$P_{WEP}(4\,823)$ — primeira colisão	50,00%	50,01%	0,01 p.p.
n_{50} WEP (mediana de K)	4 823	4 823	0,00%
E[K] WEP	5 134	5 134	-0,00%
σ_K WEP	2 683	2 682	-0,06%
n_{50} TKIP (mediana de K)	19 753 663	20 186 196	2,19%
E[K] TKIP	21 027 123	21 413 830	1,84%
$P_{TKIP}(n_{50})$	50,00%	48,80%	-1,20 p.p.
Ensaios TKIP com colisão, $n \leq 15\,000$	$5{,}96 \times 10^{-3}$	0	—
Fator n_{50} TKIP / WEP	4 095,7	4 185,4	—

Todas as diferenças entre probabilidades simuladas e exatas estão abaixo de 2 desvios-padrão (máximo: $0{,}54\sigma$), ou seja, são compatíveis com a variabilidade amostral. As discrepâncias relativas maiores aparecem no TKIP, onde o custo computacional por ensaio (≈1–2 s e ≈403 MB de memória) limitou a amostra a 500 ensaios; o erro-padrão de uma proporção próxima de 50% com esse T é de ≈2,2%.

7. Porque é que o método de Monte Carlo continua tão importante

Oito décadas depois de Los Alamos, o Monte Carlo continua a ser uma das ferramentas computacionais mais usadas na ciência e na engenharia. Este trabalho ilustra, numa escala pequena, as razões de fundo.

7.1 Razões estruturais

- **O erro não depende da dimensão.** Os métodos de quadratura determinísticos têm erro da ordem de $T^{-k/d}$ num problema de dimensão d, o que os torna impraticáveis acima de poucas dimensões (a «maldição da dimensionalidade»). O erro de Monte Carlo é $\sigma/\sqrt{T}$ qualquer que seja d. Aqui, cada ensaio TKIP é um ponto num espaço com dezenas de milhões de dimensões (um IV por pacote) — e o método funciona sem alterações.
- **Exige apenas saber simular, não saber resolver.** Bastou descrever o sistema («gerar IVs e procurar repetidos»). Se o modelo mudar — IVs não uniformes, contadores reinicializados,

várias estações com a mesma chave —, a simulação adapta-se com poucas linhas, enquanto a análise teria de ser refeita, e muitas vezes não tem solução fechada.

- **É naturalmente paralelo.** Os ensaios são independentes; o trabalho foi distribuído por 8 núcleos sem qualquer comunicação entre eles. A mesma propriedade explora hoje GPUs com milhares de núcleos e infraestruturas de nuvem.
- **Quantifica a própria incerteza.** Cada resultado vem com um intervalo de confiança, e sabe-se exatamente quanto custa melhorá-lo.
- **Valida a teoria (e vice-versa).** Como neste trabalho, a simulação é a forma mais direta de detetar erros em derivações analíticas, e a teoria é a forma de detetar erros em simuladores.

7.2 Aplicações atuais

***Tabela 9** — Áreas onde o método de Monte Carlo é hoje essencial.*

Área	Utilização do Monte Carlo
Redes e telecomunicações	Curvas de BER/BLER de esquemas de modulação e codificação; capacidade ergódica de canais MIMO; PAPR de OFDM; dimensionamento de redes e simulação de protocolos (ex.: ns-3); avaliação de sistemas 5G/6G (os Trabalhos 2, 5 e 8 desta UC usam-no).
Segurança e criptografia	Análise de colisões e de geradores aleatórios (como neste trabalho); estimação de probabilidades de sucesso de ataques; testes estatísticos de aleatoriedade.
Inteligência artificial	Monte Carlo Tree Search (base do AlphaGo, 2016); *dropout* de Monte Carlo para incerteza em redes neuronais; métodos de aprendizagem por reforço; amostragem em modelos generativos.
Estatística bayesiana	MCMC (Metropolis–Hastings, Gibbs, Hamiltonian Monte Carlo) para amostrar distribuições a posteriori em problemas com milhares de parâmetros.
Finanças e seguros	Avaliação de derivados, Value-at-Risk e testes de esforço exigidos pela regulação bancária e seguradora.
Física e medicina	Transporte de partículas (Geant4, usado no CERN), planeamento de radioterapia, dosimetria; simulação de materiais e de sistemas moleculares.
Computação gráfica	*Path tracing*: resolução da equação de renderização de Kajiya por amostragem de trajetos de luz, usada no cinema de animação e em motores de jogos com *ray tracing*.
Engenharia e clima	Análise de fiabilidade e de tolerâncias, gestão de projetos, previsão meteorológica e climática por conjuntos (*ensembles*).

7.3 Limitações e como são ultrapassadas

O método não é gratuito. A convergência $1/\sqrt{T}$ é lenta para precisões elevadas, e os acontecimentos raros — como a colisão TKIP com poucos pacotes — exigem um número de ensaios proibitivo. Por isso, a investigação continua ativa: amostragem por importância e *splitting* para acontecimentos raros, variáveis de controlo e antitéticas para reduzir a variância, sequências quasi-aleatórias (quasi-Monte Carlo, com erro próximo de $1/T$) e Monte Carlo multinível. A qualidade do gerador pseudoaleatório também é crítica, como a secção 6.6 ilustrou: este trabalho usa o Philox (Salmon et al., 2011), gerador baseado em contador com período 2^{256}, o mesmo tipo usado em GPUs (cuRAND) e no TensorFlow — e, por ironia, uma «sequência que não se repete» é exatamente a garantia que faltava ao IV do WEP.

8. Conclusões

O trabalho atingiu todos os objetivos do enunciado e respondeu às três questões de análise:

- **Q1.** Com n ≈ 4 823 pacotes, a probabilidade de colisão de IV no WEP é de 50%: 50,001% pela fórmula exata e 50,01% por simulação (200 000 ensaios), com o valor teórico dentro do intervalo de confiança.
- **Q2.** A probabilidade de colisão cresce muito mais depressa no WEP: para o mesmo número de pacotes, a razão entre as probabilidades é de $\approx 1{,}2 \times 10^7$; nenhum dos 60 000 ensaios TKIP com até 15 000 pacotes registou colisões.
- **Q3.** O TKIP necessita de 19 753 663 pacotes para os mesmos 50%: 4 096 vezes mais (2^{12}), e não 2^{24} vezes — a resistência a colisões escala com a raiz quadrada do espaço de IV.

Os dois estimadores de Monte Carlo implementados — direto e por primeira colisão — concordam entre si e com a teoria, e o estudo de convergência confirmou a lei $1/\sqrt{T}$. Do ponto de vista de segurança, a conclusão é clara: um IV de 24 bits torna a reutilização de *keystream* inevitável em segundos de tráfego, o que, combinado com a ausência de chaves por pacote, condenou o WEP. O TKIP corrigiu este problema de forma eficaz, mas a lição mais geral é a do paradoxo do aniversário: **a segurança contra colisões é metade do número de bits**.

Do ponto de vista metodológico, o trabalho mostra a complementaridade entre análise e simulação: a fórmula exata valida o simulador, o simulador valida a fórmula, e cada um cobre as limitações do outro — a simulação adapta-se a qualquer variante do modelo, e a teoria responde onde a simulação não chega, nos acontecimentos raros. É esta combinação que mantém o método de Monte Carlo como uma ferramenta de primeira linha, das redes móveis à inteligência artificial.

Referências

[1] N. Borisov, I. Goldberg e D. Wagner, «Intercepting mobile communications: the insecurity of 802.11», *Proc. 7th ACM MobiCom*, pp. 180–189, 2001.

[2] S. Fluhrer, I. Mantin e A. Shamir, «Weaknesses in the key scheduling algorithm of RC4», *Selected Areas in Cryptography (SAC 2001)*, LNCS 2259, Springer, 2001.

[3] E. Tews, R.-P. Weinmann e A. Pyshkin, «Breaking 104 bit WEP in less than 60 seconds», *Information Security Applications (WISA 2007)*, LNCS 4867, Springer, 2007.

[4] M. Beck e E. Tews, «Practical attacks against WEP and WPA», *Proc. 2nd ACM WiSec*, 2009.

[5] IEEE Std 802.11-2020, *Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications*, IEEE, 2021.

[6] N. Metropolis e S. Ulam, «The Monte Carlo method», *Journal of the American Statistical Association*, vol. 44, n.º 247, pp. 335–341, 1949.

[7] N. Metropolis, «The beginning of the Monte Carlo method», *Los Alamos Science*, n.º 15, pp. 125–130, 1987.

[8] D. E. Knuth, *The Art of Computer Programming, Vol. 1*, 3.ª ed., Addison-Wesley, 1997 (secção 1.2.11.3, função Q(n) e problema do aniversário).

[9] P. Flajolet, D. Gardy e L. Thimonier, «Birthday paradox, coupon collectors, caching algorithms and self-organizing search», *Discrete Applied Mathematics*, vol. 39, pp. 207–229, 1992.

[10] E. B. Wilson, «Probable inference, the law of succession, and statistical inference», *Journal of the American Statistical Association*, vol. 22, pp. 209–212, 1927.

[11] R. Y. Rubinstein e D. P. Kroese, *Simulation and the Monte Carlo Method*, 3.ª ed., Wiley, 2016.

[12] M. E. O'Neill, «PCG: a family of simple fast space-efficient statistically good algorithms for random number generation», Harvey Mudd College, HMC-CS-2014-0905, 2014.

[13] D. Silver *et al.*, «Mastering the game of Go with deep neural networks and tree search», *Nature*, vol. 529, pp. 484–489, 2016.

[14] J. T. Kajiya, «The rendering equation», *Proc. ACM SIGGRAPH*, pp. 143–150, 1986.

[15] J. K. Salmon, M. A. Moraes, R. O. Dror e D. E. Shaw, «Parallel random numbers: as easy as 1, 2, 3», *Proc. SC'11 (Int. Conf. for High Performance Computing)*, ACM, 2011.

[16] Enunciado dos Trabalhos laboratoriais (Python), Trabalho 6, Tecnologias de Redes de Comunicações Móveis.

Anexo A — Núcleo do estimador por primeira colisão

Excerto de `src/montecarlo.py` (código integral entregue em anexo ao relatório).

```
def primeira_colisao(bits: int, rng: np.random.Generator, fator: float = 3.0) -> int:
    """
    Gera IVs até ocorrer a primeira repetição e devolve o seu índice K (1-based).

    Em vez de inserir IVs um a um num conjunto (lento em Python), gera-se de uma
    vez um bloco de comprimento L = fator*sqrt(N), que contém uma colisão com
    probabilidade 1 - exp(-fator^2/2) (~98,9% para fator=3). Se não contiver,
    o bloco é prolongado — o resultado continua exato, só fica mais caro.

    Para localizar a primeira colisão sem um argsort completo (caro para 2^48):
      1) ordena-se uma cópia e extraem-se os valores repetidos (poucos);
      2) localizam-se as posições originais desses valores;
      3) percorrem-se essas posições por ordem: a primeira cujo valor já
         apareceu antes é a primeira colisão.
    """
    N = 2 ** bits
    L = int(fator * math.sqrt(N)) + 2
    ivs = rng.integers(0, N, size=L, dtype=np.uint64)
    while True:
        ordenados = np.sort(ivs)
        repetidos = ordenados[1:][ordenados[1:] == ordenados[:-1]]
        if repetidos.size:
            break
        ivs = np.concatenate([ivs, rng.integers(0, N, size=L, dtype=np.uint64)])

    # Há tipicamente poucos valores repetidos (~fator²/2), pelo que comparações
    # diretas são muito mais rápidas do que np.isin (que recorre a um argsort).
    mascara = np.zeros(ivs.size, dtype=bool)
    for valor in np.unique(repetidos):
        mascara |= ivs == valor
    posicoes = np.flatnonzero(mascara)
    vistos = set()
    for pos, valor in zip(posicoes.tolist(), ivs[posicoes].tolist()):
        if valor in vistos:
            return pos + 1
        vistos.add(valor)
    raise RuntimeError("inconsistência: valores repetidos sem segunda ocorrência")
```

Anexo B — Reprodução dos resultados

Requisitos: Python ≥ 3.10, NumPy ≥ 1.24 e Matplotlib ≥ 3.7 (o relatório usa ainda ReportLab). A partir da pasta do trabalho:

```
pip install -r requirements.txt
cd src
python experiencias.py              # todas as experiências (~17 min, 8 núcleos)
python experiencias.py --rapido     # versão reduzida (~3 min)
python experiencias.py --so-figuras # regenera figuras a partir de resultados/
cd ../relatorio
python gerar_relatorio.py           # gera este PDF
```

Todas as experiências usam a semente-mãe 20261031, pelo que a execução completa reproduz exatamente os valores apresentados neste relatório.